arXiv:2409.00509v2 [cs.CL] 4 Sep 2024

LongRecipe: Recipe for Efficient Long Context Generalization in Large Language Models

Zhiyuan Hu[1*] **Yuliang Liu**[2] **Jinman Zhao**[3] **Suyuchen Wang**[4] **Yan Wang**[6]
Wei Shen[7] **Qing Gu**[2] **Anh Tuan Luu**[5] **See-Kiong Ng**[1] **Zhiwei Jiang**[2] **Bryan Hooi**[1]
[1] National University of Singapore [2] Nanjing University
[3] University of Toronto [4] Mila, Québec AI Institute / Université de Montréal
[5] Nanyang Technological University [6] Tencent Inc [7] Baidu Inc

Abstract

Large language models (LLMs) face significant challenges in handling long-context tasks because of their limited effective context window size during pretraining, which restricts their ability to generalize over extended sequences. Meanwhile, extending the context window in LLMs through post-pretraining is highly resource-intensive. To address this, we introduce LongRecipe, an efficient training strategy for extending the context window of LLMs, including impactful token analysis, position index transformation, and training optimization strategies. It simulates long-sequence inputs while maintaining training efficiency and significantly improves the model's understanding of long-range dependencies. Experiments on three types of LLMs show that LongRecipe can utilize long sequences while requiring only 30% of the target context window size, and reduces computational training resource over 85% compared to full sequence training. Furthermore, LongRecipe also preserves the original LLM's capabilities in general tasks. Ultimately, *we can extend effective context window of open-source LLMs from 8k to 128k, achieving performance close to GPT-4 with just one day of dedicated training using a single GPU with 80G memory.* Our code is released at https://github.com/zhiyuanhubj/LongRecipe.

1 Introduction

LLMs are crucial for NLP and multi-modal tasks. However, they face challenges in applications involving long context, such as in-context learning (Brown et al., 2020), long document summarization (Koh et al., 2022), long-form QA (Krishna et al., 2021), and document-level retrieval (Callan, 1994). These challenges stem from their limited effective context window size during the pretraining process, posing new challenges in generalizing over long contexts.

A straightforward approach is to continually pre-train or fine-tune these models on long context input (Fu et al., 2024). However, expanding the context window usually results in a quadratic increase in computational and memory costs. According to the training setup in (Fu et al., 2024), extending the Llama-2 7B model's context window from 4k to 80k using 8 A100 GPUs (80G each) takes five days. The costs of resources and time increase significantly for larger models and more extended training periods. In addition to the methods mentioned, there are techniques aimed at extending the length of the context window more efficiently during fine-tuning, including PI (Chen et al., 2023), Yarn (Peng et al., 2024), and LongLoRA (Chen et al., 2024b). However, these techniques still require full-length fine-tuning, meaning they must fine-tune with the context of the target length, which is both memory- and time-intensive. Meanwhile, Randomized Positional Encoding Scheme (Ruoss et al., 2023) and PoSE (Zhu et al., 2023) simulate longer inputs within a fixed window by adjusting position indices, allowing LLMs that are trained on shorter contexts to be extended to longer context windows. However, randomized position embeddings in (Ruoss et al., 2023) disrupt local sentence structures by exaggerating the dependency lengths between neighboring tokens. PoSE, on the other hand, only considers two chunks to mimic the position index, consistently omitting longer dependencies in the sequence. This distortion creates a significant generalization gap in understanding token relationships across the sequence when extending LLMs to a long context window.

[*]Corresponding to: Zhiyuan Hu, zhiyuan_hu@u.nus.edu. Yuliang Liu contributes equally in this work

Preprint. Work in Progress.

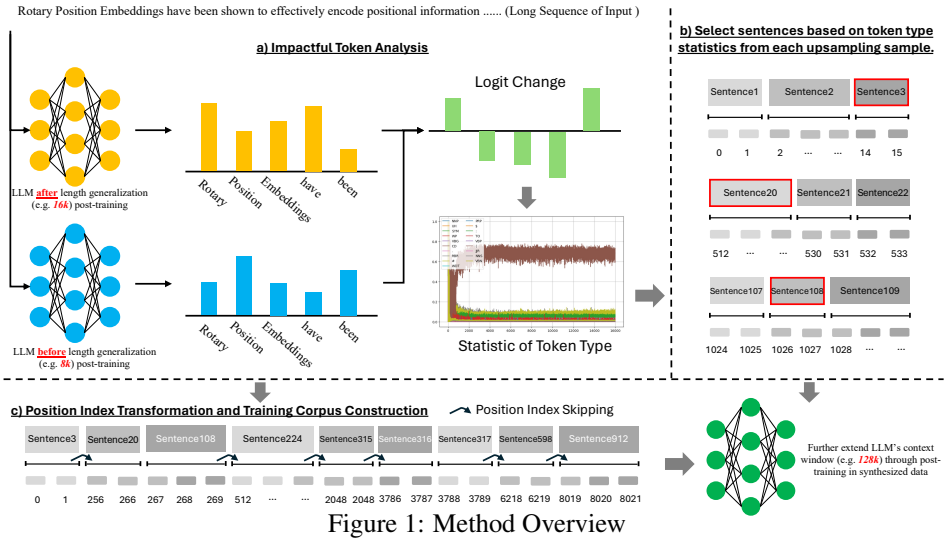

Figure 1: Method Overview

To address the aforementioned issues and further uncover the potential of efficient training for long-context generalization in LLMs, we present **LongRecipe**, an efficient framework designed to enhance long-context capabilities in models. Long context generalization depends on token distances set by position indexs, which are then combined with token representations. LongRecipe is primarily focused on optimizing the learning process by efficiently handling both position indexs and token representations. Our framework introduces Impactful Token Analysis to identify tokens that significantly influence long-text training. By focusing on these tokens, we extract shorter segments from long-text corpora, reducing text length while preserving key information. We then apply Position Index Transformation to simulate long-sequence positional indices using these shortened texts, extending the model's ability to handle long sequences without needing actual long texts. Additionally, we implement training optimizations — pretraining data replay and model merging — to enhance the model's long-text processing capabilities. As illustrated in Figure 1, LongRecipe compares the logits of output tokens from an untuned LLM with those from a tuned LLM within a longer context. This reveals significant token logit changes from long context generalization training. Sentences or paragraphs with these tokens are selected, upsampled, and segmented with continuous positional indices, then used to train the LLM, effectively extending its context window. This method efficiently captures key changes in long-context training while improving training efficiency by streamlining samples. The position index transformation also sharpens the model's understanding of long-range dependencies and sequences in extended texts.

To validate the effectiveness of LongRecipe, we conduct the empirical evaluation with Llama3-8B, Mistral-7B, Qwen2-7B on Multi-Needle In A Haystack (gkamradt, 2023), RULER (Hsieh et al., 2024), LongBench (Bai et al., 2023) and Loogle (Li et al., 2023). Applied with LongRecipe, we can extend the effective context window of an open-source LLM from 8k or 32k to 80k or 128k. The experimental results demonstrate that LongRecipe achieves an average improvement of approximately 5.5% across four metrics in three types of models, with context windows 80k and 128k. Additionally, using as little as 30% of the tokens with around 1/8 of the GPU computational resources can achieve nearly the same performance as full context window training. Currently, we can extend an open-source LLM's context window from 8k to 128k, matching GPT-4-Turbo's performance with just one day of training on a single H100 GPU. Furthermore, we test the performance of our method in general tasks, including MMLU (Hendrycks et al., 2021), GSM8K (Cobbe et al., 2021), and HumanEval (Chen et al., 2021) to assess if our method impacts LLMs' general abilities, showing it largely preserves their original performance. To summarize, our contributions are as follows:

- We introduce **LongRecipe**, leveraging impactful token analysis and position index transformation to fully uncover the potential of efficient training for long context generalization.
- LongRecipe uses training strategies of the pretraining data replay and model merging to enable LLMs to preserve the original foundational abilities and enhance long context generalization ability stably.
- Experiments conducted on context lengths from 8k or 32k to 80k or 128k of three types of LLMs validate the effectiveness of LongRecipe.

2 Preliminary

The approach that is widely used in previous pre-trained language models such as BERT (Devlin et al., 2018) is to add position embedding vectors to word embedding vectors directly. For a sequence of tokens represented as $w_1, w_2, \cdots, w_L$, with their corresponding embeddings $\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_L$, let $\mathbf{p}_1, \mathbf{p}_2, ...\mathbf{p}_L$ be absolute position embedding, the position encoding of query($\mathbf{q}$) and key($\mathbf{k}$) are $\mathbf{q}_m = W_q(\mathbf{x}_m + \mathbf{p}_m)$ and $\mathbf{k}_n = W_k(\mathbf{x}_n + \mathbf{p}_n)$. Then the unnormalized attention scores are calculated by dot-producting two vectors: $score(\mathbf{q}_m, \mathbf{k}_n) = \mathbf{q}_m^T \cdot \mathbf{k}_n$.

Rotary Position Embedding (RoPE) (Su et al., 2024) is proposed to integrate relative positional information by modulating the query and key vectors in the attention mechanism. Let D denote the dimension of hidden layers, the transformations applied are as follows:

$$\mathbf{q}_m = W_q \mathbf{x}_m \cdot e^{im\theta}, \quad \mathbf{k}_n = W_k \mathbf{x}_n \cdot e^{in\theta}, \tag{1}$$

where W_q and W_k are $|D| \times |D|$ projection matrices, m and n are the positions of the tokens, and θ is a constant that adjusts the rotation based on token positions.

$$\theta_i = 10000^{\frac{-2i}{D}}$$

RoPE operation on $\vec{q} = W_q \mathbf{x}_m$ results $\mathbf{q}_m =$:

$$\begin{bmatrix} q_0 \\ q_1 \\ \vdots \\ q_{D-2} \\ q_{D-1} \end{bmatrix} \otimes \begin{bmatrix} \cos m\theta_0 \\ \cos m\theta_0 \\ \vdots \\ \cos m\theta_{\frac{D}{2}-1} \\ \cos m\theta_{\frac{D}{2}-1} \end{bmatrix} + \begin{bmatrix} q_1 \\ q_0 \\ \vdots \\ q_{D-1} \\ q_{D-2} \end{bmatrix} \otimes \begin{bmatrix} -\sin m\theta_0 \\ \sin m\theta_0 \\ \vdots \\ -\sin m\theta_{\frac{D}{2}-1} \\ \sin m\theta_{\frac{D}{2}-1} \end{bmatrix}$$

The real part of the inner product between $\mathbf{q}_m$ and $\mathbf{k}_n$ captures the relative positional information, facilitating the model's understanding of token distances.

3 Related Works

Position Encoding Various position encoding methods have been proposed to perform extrapolation such as ALiBi (Press et al., 2022), xPos (Sun et al., 2023), KERPLE (Chi et al., 2022). RoPE (Su et al., 2024) and CoPE (Golovneva et al., 2024), the most widely used one, introduces a more complex mechanism.

Efficient Pretraining or Fine-tuning Methods Position Interpolation (PI)(Chen et al., 2023) downsizes position indices of long text to the original window size. NTK Interpolation(Peng & Quesnelle, 2023) adjusts rotation speed for small positions and linear interpolation for large ones. YaRN (Peng et al., 2024) improves NTK Interpolation with NTK-by-parts scaling to accommodate different RoPE features. Resonance RoPE (Wang et al., 2024a) refines RoPE features with integer wavelengths, improving upon YaRN for better out-of-distribution position recognition. LM-Infinite (Han et al., 2024) encodes absolute positions for starter tokens and masks middle tokens, retaining relative positions for rare tokens. Randomized Positional Embedding (Ruoss et al., 2023) simulates long text input with shorter texts by randomly selecting position indices. PoSE (Zhu et al., 2023) uses a fixed context window, dividing it into chunks with skipping bias terms, enabling adaptation to all positions within the target length. LongLoRA (Chen et al., 2024b) replaces ordinary attention with shift short attention. Temp Lora (Wang et al., 2024b) integrates context details into a temporary Lora module, incrementally trained with previously generated text. SelfExtend (Jin et al., 2024) and DCA (An et al., 2024) convert the attention computation for long sequences into chunk-based modules to achieve the training-free extension.

4 Methodology

4.1 Impactful Token Analysis

As shown in Figure 1 (a) and (b), consider a base large language model H with a context window size L. This model is further trained to extend its context window to L', resulting in a new model

denoted as H'. Using the LongRecipe methodology, we can calculate the logit offset for each token by comparing the differences between the logits produced by H and H'. We then identify the token types with the most significant changes in logits to serve as anchors for selecting sentences containing these token types, which are then used for upsampling.

Formally, for each token t, we condition both the base model H and the extended model H' on the preceding prompt $x < t$ to obtain the logit probability scores $S_H(t \mid x < t)$ and $S_{H'}(t \mid x < t)$, respectively. These scores represent the final unnormalized logits from the language modeling head over the vocabulary. The distribution of logit probability changes is then given by:

$$p(X_t \mid x < t) = \text{softmax}\left[S_{H'}(X_t \mid x < t) - S_H(X_t \mid x < t)\right] \tag{2}$$

To formally describe the process of selecting token types with the most significant logit changes, we define a significance score $\Delta(t)$ for each token type t as:

$$\Delta(t) = \sum_{i=1}^{N} \left| S_{H'}(X_t^{(i)} \mid x < t^{(i)}) - S_H(X_t^{(i)} \mid x < t^{(i)}) \right| \tag{3}$$

where N represents the total number of samples. We then rank all token types by their significance score $\Delta(t)$, and select the token types with the highest scores as anchors. The selected tokens are used to identify and upsample sentences that contain these tokens.

Intuitively, we aggregate the distributions across all samples to derive the statistics of token types whose logit probability changes are most significant. We select the top 20% of tokens based on their significance scores $\Delta(t)$ at each position (e.g., the i-th token in the sample). We then calculate the frequency of each token type (part-of-speech).

Subsequently, for a given sample, we first remove sentences that do not contain these token types, which generally constitute a significant portion of the total sentences. Then, from the remaining sentences, we select a fixed number of tokens to use for further training.

4.2 Position Index Transformation

Refer to Figure 1 (c), we aim to utilize the current data with context window L to enable the model with larger input context length $\hat{L}$ by further continual pretraining in the data with synthesized position indices. Let S be the original sequence. We define a function **seg** : $S \rightarrow \{s_1, s_2, \ldots, s_N\}$ that partitions S into N segments, where each segment s_i can be either a sentence or a paragraph, for $1 \leq i \leq N$. The function **seg** satisfies the following conditions:

$$S = s_1 \cup s_2 \cup \cdots \cup s_N \tag{4}$$

The union of all segments reconstructs the original sequence and segments are disjoint:

$$s_i \cap s_j = \emptyset \quad \text{for all} \quad i \neq j \tag{5}$$

To vary the spacing between each segment, we will randomly skip some position indices from 0 to M, where M is a parameter of our method. When $M = 0$, the position indices of the two segments will be continuous.

We start by defining **pos**(s_i) as the position index of the first token of segment s_i. For each segment, the position indices are sequentially increased by 1 for each token within that segment. The position index of the first token in the first segment is set to 0, i.e., **pos**$(s_1) = 0$.

For subsequent segments, we introduce a random skip represented by a function $g(s_i)$ which takes values from $0, 1, \ldots, M$. This function represents the gap before the start of segment s_i and is determined randomly for each segment. Thus, the position index of the first token of segment s_i, for $i \geq 2$, can be defined recursively as follows:

$$\text{pos}(s_i) = \text{pos}(s_{i-1}) + |s_{i-1}| + g(s_i) + 1 \tag{6}$$

Where $|s_{i-1}|$ represents the number of tokens in segment s_{i-1}. We repeat this process until the position index of the last token of the last segment s_N does not exceed $\hat{L}$.

To achieve comprehensive coverage of the target context window, we re-sample both the length and skipping term of every chunk for each training example.

4.3 Training Optimization Strategies

When we extend the effective context window of LLMs, we also want to enable the LLMs with strong general abilities within their original context window. Therefore, we explore the below two training optimization strategies to achieve that.

Pretraining Data Replaying In this module, we address the issue of maintaining a model's general capabilities during post-training by employing a Pretraining Data Replay strategy. Specifically, we define two datasets: $\mathcal{D}_1$, which represents the original pretraining data, and $\mathcal{D}_2$, which is a replay dataset derived from $\mathcal{D}_1$. Both $\mathcal{D}_1$ and $\mathcal{D}_2$ share the same distribution.

The replay dataset $\mathcal{D}_2$ is used for further training after the model undergoes long-sequence extension training. This process is intended to help the model recover and reinforce its general capabilities that may have been affected during the length extension training. Formally, during the replay phase, the model is trained on $\mathcal{D}_2$ to restore and enhance its generalization abilities: $\Theta_{\text{replay}} = \text{Train}(\Theta_{\text{extended}}, \mathcal{D}_2)$. Here, Θ_{extended} represents the model after it has undergone long-sequence extension training, and Θ_{replay} is the model after the replay phase using $\mathcal{D}_2$.

Model Merging To maintain the general abilities of original LLMs trained in short context window, we utilize a model merging technique to integrate the capabilities of two distinct models: one that is the original model without context window extension ($\Theta^{(o)}$) and another that is trained with longer context and pretraining data replaying ($\Theta^{(replay)}$). We use two hyperparameters λ_1 and λ_2 to retains the general abilities of original models and the long context generalization of tuned model. The merged model can be represented by the following equation:

$$\Theta_{\text{merge}} = \lambda_1 \Theta^{(o)} + \lambda_2 \Theta^{(replay)} \tag{7}$$

5 Experimental Setup

5.1 Baselines

Full-length Text Training (FLT). We train the LLMs using a corpus that contains the full target context length. This approach serves as a baseline for comparing the performance and observing any potential loss when applying our method.

Randomized Positional Encoding Scheme (RPES) (Ruoss et al., 2023) simulates the positions of longer sequences and randomly selects an ordered subset to match longer length.

Positional Skip-wisE (PoSE) (Zhu et al., 2023) simulates long inputs using a fixed context window. It divides the original context window into two chunks and applies distinct skipping bias terms to manipulate the position indices of each chunk. These bias terms and the lengths of the chunks are changed for each training example, enabling model to adapt to all positions within the target length.

5.2 Dataset and Evaluation

Dataset for Training We use the dataset in the work (Fu et al., 2024) as training set. The dataset derived from SlimPajama (Cerebras, 2023), incorporates domain balancing and length upsampling. This dataset includes 80k samples and 128k tokens for each, we use 10k samples in the experiments for all baselines.

Benchmarks of Long Context Generalization The **Needle In A Haystack (NIAH)** framework (gkamradt, 2023) tests LLMs' ability to retrieve hidden information by embedding a "needle" (fact) within a "haystack" (long document). As the current LLMs can almost perform perfectly in single-needle retrieval task, we use more challenging multi-needle retrieval task to evaluate LLMs' ability, namely NIAH(M). **RULER** (Hsieh et al., 2024) offers flexible sequence lengths and task complexities with 13 sub-task categories, including retrieval and question answering. **LongBench** (Bai et al., 2023) is the first bilingual benchmark for long context understanding, featuring 21 tasks in six categories. We supplement more details about these benchmark in Appendix D.

Datasets for Assessment of Fundamental Abilities of LLMs We use three benchmarks to test if the continual pretraining process affects LLMs' fundamental abilities within their original context length. **MMLU** covers 57 subjects across STEM, the humanities, the social sciences, philosophy, law, medicine and more (Hendrycks et al., 2021). **GSM8K** (Cobbe et al., 2021) is a benchmark of math problems. **HumanEval** (Chen et al., 2021) is a code problem solving dataset.

5.3 Setup

Long Context Training We train the LLMs using samples with 30% of the extended context window length and optimize efficiency with FlashAttention 2 (Dao et al., 2022) and DeepSpeed Zero 3 (Aminabadi et al., 2022). To further train the LLMs with longer context window, we utlize Accelerator of Huggingface (Face) and Sequence Parallel technique (e.g. DeepSpeed-Ulysses (Jacobs et al., 2023) and Ring Attention (Liu et al., 2023; Zhu)) to optimize the GPU memory demands. More details including RoPE scaling, Batch Size, Hours to Train and others are in Appendix A.

Pretraining Data Replay We use WizardLM-evol-instruct-70k (Luo et al., 2023) and Magicoder-OSS-Instruct-75K(Wei et al., 2024), totally with 68M tokens. Based on the findings in (Yang et al., 2024b), replaying 5% to 10% of the post-training dataset is considered the optimal configuration. For our setup, we use a batch size of 96, a learning rate of 5e-6, and a decay rate of 0.1. **Model Merging** We set λ_1 and λ_2 as 0.5, hence it would be the average weight for model merging.

LLMs We test various LLMs to evaluate our approach, including Llama3 (Meta.AI, 2024), Mistral (Mistral.AI, 2024), Qwen2 (Yang et al., 2024a), GPT-4 (OpenAI: Josh Achiam et al., 2023), Gemini-1.5-Pro (Reid et al., 2024) and others. Information about all models is in Appendix B.

6 Experimental Performance

Model	Length	Method	Long Context Generalization			General Abilities		
			NIAH(M)	RULER	LongBench	MMLU	GSM8K	HumanEval
Llama3-8B-I	8k	Base Model	-	-	-	**65.7**	**71.4**	**37.5**
	80k	FLT	82.3	75.7	28.1	62.2	54.5	32.7
		RPES	71.8	71.4	**27.9**	61.4	53.1	15.4
		PoSE	68.8	69.9	27.2	62.6	58.2	25.6
		LongRecipe	**78.9**	**74.5**	26.9	63.0	57.9	29.3
	128k	FLT	73.2	75.8	26.4	58.3	50.9	16.5
		RPES	72.7	71.5	**27.3**	59.2	46.0	16.8
		PoSE	80.1	75.3	26.7	61.9	51.1	21.1
		LongRecipe	**82.6**	**76.0**	25.5	62.1	54.9	24.2
Mistral-7B	32k	Base Model	-	-	-	**55.7**	**28.4**	**31.1**
	80k	FLT	43.0	57.4	17.7	52.6	25.2	25.6
		RPES	60.4	65.1	**25.6**	51.8	27.4	24.7
		PoSE	64.7	65.0	23.6	54.9	29.4	27.6
		LongRecipe	**64.7**	**67.2**	22.5	53.7	28.0	27.6
	128k	RPES	41.9	52.5	21.8	52.8	26.5	24.8
		PoSE	35.9	46.3	22.3	53.4	25.9	22.6
		LongRecipe	**53.4**	**58.2**	**23.7**	53.1	26.0	24.2
Qwen2-7B	32k	Base Model	-	-	-	66.1	58.3	20.3
	80k	FLT*	64.7	69.5	28.2	**68.4**	**63.1**	**27.4**
		RPES	73.7	68.9	27.8	65.7	55.1	16.0
		PoSE	70.0	66.7	**28.2**	66.6	58.9	17.7
		LongRecipe	**79.5**	**70.8**	25.7	65.7	57.2	19.1
	128k	FLT*	52.7	51.3	28.2	**68.4**	**63.1**	**27.4**
		RPES	64.6	64.6	**26.8**	65.5	56.1	14.8
		PoSE	58.5	60.1	27.7	67.1	58.2	20.9
		LongRecipe	**65.8**	**64.8**	26.2	65.9	58.7	17.3
Other LLMs	128k	Llama3.1-8B	72.0	69.8	24.5	62.0	41.8	38.4
		Yi-9B-200k	65.7	62.3	30.3	42.5	51.3	21.3
		Yi-34B-200k	84.9	77.3	29.1	76.3	67.2	23.2
		Qwen2-7B-Instruct	38.8	52.5	31.9	69.5	55.6	43.3
		Gradient-Llama3-8B	89.6	78.4	27.8	59.4	49.9	13.4
		Llama3.1-8B-Instruct	89.0	77.7	30.0	73.0	84.5	72.6
		GLM4-9B-Chat-1M	90.2	79.9	29.4	74.7	84.0	70.1
		Llama3.1-70B-Instruct	68.3	66.6	42.8	86.0	95.1	80.5
		Qwen2-72B-Instruct	83.4	53.7	31.0	84.2	89.5	64.6
		Gradient-Llama3-70B	79.2	72.1	31.8	72.5	73.4	33.5
		GPT-4	76.2	81.2	50.0	80.5	93.0	73.2
		Gemini-1.5-Pro	82.0	94.4	48.5	81.9	91.7	71.9

Table 1: Performance of different methods in long context generalization tasks and general abilities benchmarks. FLT* in Qwen2-7B denotes the Qwen2-7B base model combined with YARN and DCA methods for targeting the context window, as detailed in their technical report. In 'Other LLMs' part, the models above dashed line are the base model and blow are instruction tuned models. All the experiment results of FLT, RPES and PoSE are implemented by us.

6.1 Long Context Generalization

The LongRecipe method shows an average improvement of 6.6% over RPES and 7.8% over PoSE in the NIAH(M) task. In the RULER evaluation, LongRecipe outperforms RPES by 2.9% and PoSE by 4.7%. Especially, In the NIAH task, Llama3-8B-I (80k) shows a 10.1% improvement with LongRecipe over PoSE. In the RULER task, Mistral-7B (128k) improves by 11.9%.

Compared to the performance of current closed-source and open-source LLMs with a 128k context window, LongRecipe not only surpasses base models like Yi-9B, Llama3.1-8B, and the instruction model Qwen2-7B-Instruct but also achieves performance comparable to Gradient-Llama3-8B, which uses four times the tokens and full-length training. Additionally, LongRecipe approaches the performance levels of GPT-4.

6.2 Maintaining General Abilities

Table 1 shows that LLMs can nearly maintain their general abilities with short inputs, as seen by the minor performance drop in MMLU. Despite some remaining gaps in mathematical (GSM8K) and programming (HumanEval) abilities, the model merging and pretraining data replay strategy successfully restored approximately 75% and 65% of the original capabilities.

6.3 Ablation Study

Benefits of Impactful Token Analysis and Position Index Transformation As shown in Table 2, the performance will drop significantly in NIAH(M) and RULER metrics when we randomly select sentence from long sequence (LongRecipe (Random T)) rather than using analyzed token pattern. Additionally, the application of Position Index Transformation can bring average 3.3% improvement from PoSE to LongRecipe (w/o T).

Method	NIAH(M)	RULER	LongBench
PoSE	68.8	69.9	27.2
LongRecipe (w/o T)	71.9	71.7	29.2
LongRecipe (Random T)	70.1	69.8	27.1
LongRecipe	78.9	74.5	26.4

Table 2: Performance of different ablation setings, LongRecipe (w/o T) uses the short exiting samples as PoSE and apply position index transformation on it. LongRecipe (Random T) select the sentence randomly from long sequence of sample and construct a new short samples. All experiments are based on Llama3-8B-instruct and we use 30% tokens of 80k target context window.

Effect of Pre-training Data Replay and Model Merging for Maintaining General Abilities In Table 3, comparing the models before and after replaying shows noticeable improvement, particularly on the GSM8K dataset. By further merging with the original model, we can enhance the model's general capabilities, as seen in the MMLU performance (63% vs. 65.7%). Although there are still some gaps in mathematical (GSM8K) and coding (HumanEval) capabilities, the model merging and pretraining data replaying successfully recovers approximately 75% and 65% of the original abilities.

Method	**Before Replaying**			**After Replaying**			**After Model Merging**		
	MMLU	GSM8K	HE	MMLU	GSM8K	HE	MMLU	GSM8K	HE
FLT	58.1	39.7	20.9	58.2	47.2	17.5	62.2	54.5	32.3
RPES	54.0	33.6	15.2	59.7	46.7	3.3	61.8	53.9	12.6
PoSE	58.1	39.1	17.0	60.7	49.5	5.7	62.6	58.2	25.6
LongRecipe	58.6	42.7	20.1	62.1	50.9	6.7	63.0	57.9	29.3

Table 3: Performance of different stages in long context generalization training, pretraining data replaying and model merging. HE represents HumanEval. All experiments are conducted using the Llama3-8B-instruct model, with 30% of tokens utilized within an 80k token target context window.

Performance Comparison Based on Various Number of Tokens

As the number of tokens per sample increases, the performance of each sample improves consistently. However, the benefit gained from increasing the number of tokens (i.e., extending the context length) diminishes. Even we increase the token ratio from 30% to 100%, only around 1% improvement can be obtained. This is particularly evident in the results of Llama3-8B for a 128-token context window, as shown in Table 1, where we achieve even better performance than FLT with 100% of the tokens.

Ratio	NIAH(M)	RULER	LongBench
10%	65.3	67.4	29.6
20%	70.9	70.7	28.8
30%	78.9	74.5	26.9
40%	72.0	71.0	25.7
100%	82.3	75.7	28.1

Figure 2: We conduct context window extension experiments using Llama3-8B-I with an 80k token length. Starting from 10%, which represents 8k tokens per sample, 20% corresponds to 16k tokens, 30% to 24k tokens, and 40% to 32k tokens. The 100% configuration utilizes entire long sample.

6.4 Analysis

Distance Among Tokens and Continual Length of Segments We suppose the effectiveness of position index transformation stems from improving distances among token while maintaining local information via continual segment. Therefore, we calculated the average distance among tokens and the average continuous segment length for different methods.

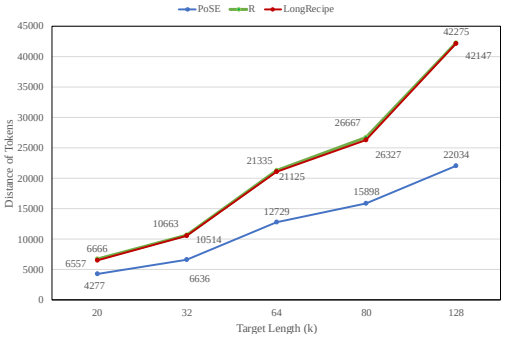

Figure 3: Comparison of average distance among tokens for different methods and context window.

As shown in Figure 3, the LongRecipe approach achieves approximately twice the token distance compared to PoSE in 128k setting. Additionally, LongRecipe maintains an average continuous segment length of 88, which helps the LLM recognize local dependency structures. In contrast, the average continuous segment length with RPES is nearly 0, greatly disrupting local sentence structures.

Analysis of Token Type In the LongRecipe approach, we first compare the change in token logits before and after tuning the long context window. We then select the top 20% of tokens that exhibit

This chart displays the frequency distribution and relative relationships of parts of speech for tokens with significant logits changes across different positions in the text. NUM (numerals) has the highest frequency, stabilizing around 0.4 throughout the text. In contrast, other parts of speech have significantly lower frequencies. For example, PRON (pronouns) and AUX (auxiliary verbs) have frequencies around 0.15, while CCONJ (conjunctions) and ADP (adpositions) have frequencies around 0.1. The frequency of NUM is approximately 2.67 times that of PRON and AUX and about 4 times that of CCONJ and ADP. These findings suggest that long-context tuning has varying effects on different token types, which further reinforces the motivation behind our method.

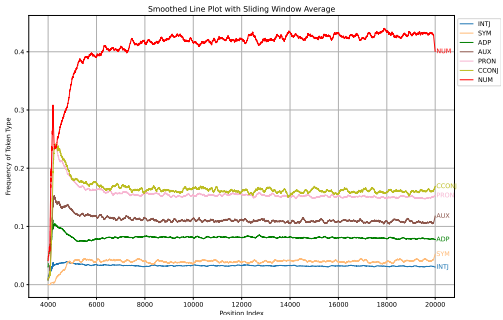

Figure 4: Frequency Distribution of Parts of Speech for Tokens with Significant Logits Changes Across Text Positions. INTJ (Interjection), SYM (Symbol), ADP (Adposition), AUX (Auxiliary), PRON (Pronoun), CCONJ (Conjunction), NUM (Numeral)

the most significant change at each index. These selected tokens are further analyzed for their part of speech distribution patterns. The results of this analysis are presented in Figure 4.

Do Coherence and Cohesion Matter in Long Context Generalization? In this work, we select sentences from long samples based on analyzed token patterns, which may impact semantic coherence and cohesion. However, our current results based on LongRecipe can match or even surpass those from some full-length samples, suggesting that coherence and cohesion may not be as critical for long-context training. To further investigate this, we utilize the Long Dependency Score (Chen et al., 2024a) to assess the long dependencies in different datasets, which may be more crucial for long context training. After calculation, PoSE and RPES, which use the same existing short samples, achieved a score of 12.07, while the data constructed by LongRecipe and that concatenated from several short documents in FLT scored 17.88. Since the data used in FLT is concatenated rather than naturally occurring long context, the semantic quality is not satisfactory. LongRecipe does not significantly harm the long dependencies required for long-context training, even though it may influence coherence and cohesion to some extent.

Furthermore, during the pretraining process, the model primarily focuses on learning grammar and semantics. However, in the post-training phase on long texts, the model has already acquired grammar and semantic knowledge, so the focus may shift to capturing long dependencies among tokens. At this stage, it is possible to ignore certain tokens that are less important. These sentence might significantly influence overal semantic but contribute little to the learning of more complex attention patterns across longer sequences. Although this approach may slightly affect the model's general capabilities, the impact is minimal, and the model can quickly recover. More importantly, we can leverage LongRecipe method to achieve the efficient training for long context training.

7 Conclusion

In this work, we presented LongRecipe, a novel and efficient framework for extending the context window of LLMs to enhance their performance on long-context tasks. By integrating impactful token analysis, position index transformation, and training strategies, LongRecipe effectively simulates long-sequence inputs while maintaining training efficiency. Our extensive experiments on various LLMs, with extended context windows in 80k to 128k, demonstrated that LongRecipe could achieve substantial improvements in long-context generalization with significantly reduced computational resources. Notably, the method requires only 30% of the target context window size and cuts down training costs over 85% compared to full-length post-training. Moreover, LongRecipe preserves the original capabilities of the LLMs in general tasks, ensuring a balanced enhancement of both long-range dependency understanding and foundational model performance.

8 Limitation and Onging Work

Supervised Fine-Tuning (SFT) While our current post-training approach, based on instruction or base models, yields satisfactory performance in NIAH and RULER, the absence of SFT still creates a gap between our method and the state-of-the-art (SOTA) LLMs. Recently, the release of LongWriter (Zhipu, 2024) for long-context SFT presents a promising option for further enhancing our fine-tuning process.

Longer Context Generalization The latest LLMs have pushed the boundaries of long-context capabilities to handle up to 1 million tokens, enabling users to input vast amounts of data. We plan to train and release models with 512k and 1M token capacities using the effective training strategies outlined in LongRecipe. This approach will further enhance the generalization of our method.

References

Gradient AI. Gradient ai, llama3 series, 2024. URL https://huggingface.co/gradientai.

Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15. IEEE, 2022.

Chenxin An, Fei Huang, Jun Zhang, Shansan Gong, Xipeng Qiu, Chang Zhou, and Lingpeng Kong. Training-free long-context scaling of large language models. *arXiv preprint arXiv:2402.17463*, 2024.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

James P Callan. Passage-level evidence in document retrieval. In *SIGIR'94: Proceedings of the Seventeenth Annual International ACM-SIGIR Conference on Research and Development in Information Retrieval, organised by Dublin City University*, pp. 302–310. Springer, 1994.

Cerebras. Slimpajama: A 627b token, cleaned and deduplicated version of redpajama, 2023. URL https://cerebras.ai/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama.

Longze Chen, Ziqiang Liu, Wanwei He, Yunshui Li, Run Luo, and Min Yang. Long context is not long at all: A prospector of long-dependency data for large language models. *arXiv preprint arXiv:2405.17915*, 2024a.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.

Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. LongloRA: Efficient fine-tuning of long-context large language models. In *The Twelfth International Conference on Learning Representations*, 2024b. URL https://openreview.net/forum?id=6PmJoRfdaK.

Ta-Chung Chi, Ting-Han Fan, Peter J Ramadge, and Alexander Rudnicky. Kerple: Kernelized relative positional embedding for length extrapolation. *Advances in Neural Information Processing Systems*, 35:8386–8399, 2022.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168, 2021. URL https://arxiv.org/abs/2110.14168.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Hugging Face. Accelerate. `https://huggingface.co/docs/accelerate/en/package_reference/accelerator`.

Yao Fu, Rameswar Panda, Xinyao Niu, Xiang Yue, Hannaneh Hajishirzi, Yoon Kim, and Hao Peng. Data engineering for scaling language models to 128k context. *arXiv preprint arXiv:2402.10171*, 2024.

gkamradt. Llmtest_needleinahaystack: Doing simple retrieval from llm models. `https://github.com/gkamradt/LLMTest_NeedleInAHaystack/tree/main`, 2023. [Online; accessed 29-December-2023].

Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Diego Rojas, Guanyu Feng, Hanlin Zhao, Hanyu Lai, Hao Yu, Hongning Wang, Jiadai Sun, Jiajie Zhang, Jiale Cheng, Jiayi Gui, Jie Tang, Jing Zhang, Juanzi Li, Lei Zhao, Lindong Wu, Lucen Zhong, Mingdao Liu, Minlie Huang, Peng Zhang, Qinkai Zheng, Rui Lu, Shuaiqi Duan, Shudan Zhang, Shulin Cao, Shuxun Yang, Weng Lam Tam, Wenyi Zhao, Xiao Liu, Xiao Xia, Xiaohan Zhang, Xiaotao Gu, Xin Lv, Xinghan Liu, Xinyi Liu, Xinyue Yang, Xixuan Song, Xunkai Zhang, Yifan An, Yifan Xu, Yilin Niu, Yuantao Yang, Yueyan Li, Yushi Bai, Yuxiao Dong, Zehan Qi, Zhaoyu Wang, Zhen Yang, Zhengxiao Du, Zhenyu Hou, and Zihan Wang. Chatglm: A family of large language models from glm-130b to glm-4 all tools, 2024.

Olga Golovneva, Tianlu Wang, Jason Weston, and Sainbayar Sukhbaatar. Contextual position encoding: Learning to count what's important. *arXiv preprint arXiv:2405.18719*, 2024.

Chi Han, Qifan Wang, Hao Peng, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. Lm-infinite: Zero-shot extreme length generalization for large language models, 2024.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023.

Hongye Jin, Xiaotian Han, Jingfeng Yang, Zhimeng Jiang, Zirui Liu, Chia-Yuan Chang, Huiyuan Chen, and Xia Hu. Llm maybe longlm: Self-extend llm context window without tuning. *arXiv preprint arXiv:2401.01325*, 2024.

Huan Yee Koh, Jiaxin Ju, Ming Liu, and Shirui Pan. An empirical survey on long document summarization: Datasets, models, and metrics. *ACM computing surveys*, 55(8):1–35, 2022.

Kalpesh Krishna, Aurko Roy, and Mohit Iyyer. Hurdles to progress in long-form question answering. *arXiv preprint arXiv:2103.06332*, 2021.

Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. Loogle: Can long-context language models understand long contexts? *arXiv preprint arXiv:2311.04939*, 2023.

Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*, 2023.

Meta.AI. Llama 3.1 model card, 2024. URL https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md.

Mistral.AI. La plateforme, 2024. URL https://mistral.ai/news/la-plateforme/.

OpenAI: Josh Achiam et al. GPT-4 technical report. *arXiv:2303.08774*, 2023.

Bowen Peng and Jeffrey Quesnelle. Ntk-aware scaled rope allows llama models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation. https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/ntkaware_scaled_rope_allows_llama_models_to_have, 2023.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=wHBfxhZu1u.

Ofir Press, Noah Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=R8sQPpGCv0.

Machel Reid et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv:2403.05530*, 2024.

Anian Ruoss, Grégoire Delétang, Tim Genewein, Jordi Grau-Moya, Róbert Csordás, Mehdi Bennani, Shane Legg, and Joel Veness. Randomized positional encodings boost length generalization of transformers. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1889–1903, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-short.161. URL https://aclanthology.org/2023.acl-short.161.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. ISSN 0925-2312. doi: https://doi.org/10.1016/j.neucom.2023.127063. URL https://www.sciencedirect.com/science/article/pii/S0925231223011864.

Yutao Sun, Li Dong, Barun Patra, Shuming Ma, Shaohan Huang, Alon Benhaim, Vishrav Chaudhary, Xia Song, and Furu Wei. A length-extrapolatable transformer. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14590–14604, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.816. URL https://aclanthology.org/2023.acl-long.816.

Suyuchen Wang, Ivan Kobyzev, Peng Lu, Mehdi Rezagholizadeh, and Bang Liu. Resonance RoPE: Improving context length generalization of large language models. In *Findings of the Association for Computational Linguistics ACL 2024*, pp. 586–598, Bangkok, Thailand and virtual meeting, August 2024a. Association for Computational Linguistics. URL https://aclanthology.org/2024.findings-acl.32.

Y. Wang, D. Ma, and D. Cai. With greater text comes greater necessity: Inference-time training helps long text generation, 2024b.

Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Empowering code generation with oss-instruct. In *Forty-first International Conference on Machine Learning*, 2024.

Thomas Wolf et al. Huggingface's Transformers: State-of-the-art natural language processing. *arXiv:1910.03771*, 2019.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024a.

Enneng Yang, Li Shen, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. Model merging in llms, mllms, and beyond: Methods, theories, applications and opportunities. *arXiv preprint arXiv:2408.07666*, 2024b.

Alex Young et al. Yi: Open foundation models by 01.AI. *arXiv:2403.04652*, 2024.

Zhipu. Longwriter, 2024. URL `https://huggingface.co/datasets/THUDM/LongWriter-6k`.

Dawei Zhu, Nan Yang, Liang Wang, Yifan Song, Wenhao Wu, Furu Wei, and Sujian Li. Pose: Efficient context window extension of llms via positional skip-wise training. *arXiv preprint arXiv:2309.10400*, 2023.

Zilin Zhu. Zigzag-ring-attention. `https://github.com/zhuzilin/ring-flash-attention`.

A Training Setup

Model	**Llama3-8B**		**Qwen2-7B**	
Extended Context Length	80k	128k	80k	128k
Training Sample Length	24k	38.4k	24k	38.4k
RoPE scaling (Dynamic NTK)	48.9M	131.5M	13.5M	13.5M
RoPE factor (Dynamic NTK)	10	16	4	4
Batch Size	96	96	96	96
Steps	104	104	104	104
Total Tokens	240M	384M	240M	384M
Learning Rate	5e-5	5e-5	5e-5	5e-5
# GPUs and Type	1×A800/H100	2×A800/H100	1×A800/H100	1×A800/H100
Total GPU Memory	56G	104G	64G	72G
Total CPU Memory	148G	172G	168G	208G
Hours to Train	26/16	30/20	23/15	44/28

Table 4: Training Configuration Details

B Models

We select in total 15 models for evaluation and analysis. We assess two commercial close-source GPT-4 and Gemini-1.5, and 13 open-source models.

Model	Size	Context Length	Huggingface (Wolf et al., 2019) / API
GPT-4 (OpenAI: Josh Achiam et al., 2023)	-	128K	`gpt-4-1106-preview`
Gemini-1.5-Pro (Reid et al., 2024)	-	1M	`gemini-1.5-pro`
Llama3-8B-I (Meta.AI, 2024)	8B	8K	meta-llama/Meta-Llama-3-8B-Instruct
Llama3.1-8B (Meta.AI, 2024)	8B	128K	meta-llama/Meta-Llama-3.1-8B
Llama3.1-8B-Instruct (Meta.AI, 2024)	8B	128K	meta-llama/Meta-Llama-3.1-8B-Instruct
Llama3.1-70B-Instruct (Meta.AI, 2024)	70B	128K	meta-llama/Meta-Llama-3.1-70B-Instruct
Qwen2-7B (Yang et al., 2024a)	7B	128K	Qwen/Qwen2-72B-Instruct
Qwen2-7B-Instruct (Yang et al., 2024a)	7B	128K	Qwen/Qwen2-7B-Instruct
Qwen2-7B-Instruct (Yang et al., 2024a)	72B	128K	Qwen/Qwen2-72B-Instruct
Yi-9B-200k (Young et al., 2024)	9B	200K	01-ai/Yi-34B-200K
Yi-34B-200k (Young et al., 2024)	34B	200K	01-ai/Yi-34B-200K
Mistral-7B (Mistral.AI, 2024)	7B	32K	mistralai/Mistral-7B-Instruct-v0.3
GLM4-9B-Chat-1M (GLM et al., 2024)	9B	1M	THUDM/glm-4-9b-chat-1m
Gradient-Llama3-8B (AI, 2024)	8B	1M	gradientai/Llama-3-8B-Instruct-Gradient-1048k
Gradient-Llama3-70B (AI, 2024)	70B	1M	gradientai/Llama-3-70B-Instruct-Gradient-1048k

Table 5: Information of evaluated and analyzed models in.

C Pseudo Code for Position Index Transformation

D Details about Long Context Benchmarks

NIAH (M) and RULER: For NIAH (M), we report the average score across three tasks in RULER: *niah_multikey*, *niah_multivalue*, and *niah_multiquery*. For RULER, we present the average score for all 13 subsets with Llama3-8B-Instruct and Mistral-7B-v0.3, and the average score for 12 subsets (excluding *Variable Tracking*) with Qwen2-7B.

LongBench: For LongBench, we report the average score across all 21 subsets for the models.

Algorithm 1 Position Index Transformation

Initialize:
Initialize source length L_s and target length L_t
Load dataset $\mathcal{D}$ with each sample having length L_s
Position Index Transformation:
for each sample S in $\mathcal{D}$ **do**
    Split S into N sentences based on delimiters '. ! ? \n'
    Initialize a list $\mathcal{L}$ of length L_t, filled with zeros
    Randomly select $N-1$ distinct positions in $\mathcal{L}$
    Insert the first sentence at position 0 and each of the remaining sentences at the selected $N-1$ positions in $\mathcal{L}$
    Flatten $\mathcal{L}$ by removing zeros, and the indices of the non-zero elements represent the new position indexes
end for
Save New Position Indexes